



Challenge Sprint 3 – MEDIX

Disruptive Architectures: IoT, Big Data & AI

FIAP — Challenge em parceria com Oracle · 2026

Sumário

1. Integrantes do Grupo.....	
2. Problema a Ser Resolvido	
3. Objetivos da Solução.....	
4. Componente de IA Escolhido	
5. Justificativa Técnica do Modelo	
6. Caso de Uso no Oracle APEX	
7. Estratégia de Dados	
8. Diagrama de Arquitetura.....	
9. Alinhamento com os Critérios de Avaliação	
10. Tecnologias Utilizadas.....	
11. Próximos Passos.....	
12. Repositórios GitHub.....	

1. Integrantes do Grupo

- RM 561061 – Arthur Thomas Mariano de Souza
- RM 559873 – Davi Cavalcanti Jorge
- RM 559728 – Mateus da Silveira Lima

2. Problema a Ser Resolvido

O ecossistema de saúde enfrenta uma lacuna crítica de informação e inteligência operacional que afeta tanto pacientes quanto instituições. Para o paciente, a busca por atendimento é um processo fragmentado e ineficiente: não há acesso centralizado a informações sobre disponibilidade de especialidades, tempo de espera ou capacidade das unidades, o que leva a peregrinações desnecessárias e à superlotação de certos serviços.

Para hospitais e clínicas, o problema é igualmente grave. A gestão de recursos ocorre de forma reativa, os gestores só percebem que estão com equipes sobrecarregadas, falta de leitos ou filas longas depois que o caos já está instalado. A ausência de ferramentas preditivas inteligentes resulta em três consequências diretas:

- Sobrecarga de colaboradores em períodos de alta demanda não antecipada, comprometendo a qualidade do atendimento.
- Ociosidade de recursos e custos operacionais elevados em momentos de baixa demanda não prevista.
- Deterioração progressiva da experiência do paciente, afetando a reputação e os resultados das unidades parceiras.

Este problema não é periférico ao projeto Medix, ele está no centro da proposta de valor da plataforma. A documentação do projeto já declara explicitamente o objetivo de 'utilizar ferramentas de análise de dados para prever picos de demanda'. Esta sprint trata da arquitetura que viabiliza esse objetivo por meio de Inteligência Artificial.

3. Objetivos da Solução

A plataforma Medix ataca esse problema com três objetivos estratégicos centrais:

- Empoderar o Paciente: disponibilizar um aplicativo móvel para que pacientes localizem unidades de saúde, verifiquem a disponibilidade de recursos e especialidades e tomem decisões de atendimento de forma informada e ágil.
- Otimizar a Gestão Hospitalar: fornecer às instituições de saúde uma plataforma para gerenciar recursos em tempo real, analisar o fluxo de pacientes e utilizar ferramentas de análise de dados e inteligência artificial para prever picos de demanda com antecedência suficiente para ação preventiva.
- Criar um Ecossistema Integrado: construir a ponte tecnológica que conecta a demanda dos pacientes à capacidade das instituições, melhorando a eficiência operacional e a qualidade do atendimento em saúde de forma sistêmica.

Nesta Sprint 3, o foco está no segundo objetivo, especificamente no componente de Inteligência Artificial que viabiliza a previsão de demanda dentro do ambiente Oracle APEX e Oracle Cloud Infrastructure.

4. Componente de IA Escolhido

Após análise dos dados disponíveis no domínio do projeto e dos requisitos operacionais das unidades de saúde parceiras, o componente de IA escolhido para esta sprint é:

Componente

Previsão de Demanda de Atendimentos com Modelo de Séries Temporais

Modelo Principal

Prophet — biblioteca open source desenvolvida pela Meta (Facebook), especializada em séries temporais com sazonalidade múltipla.

Modelo Alternativo

Gradient Boosting (XGBoost / LightGBM) — utilizado quando o volume de dados históricos é suficiente para treinamento supervisionado com maior precisão preditiva.

Ambiente de Execução

OCI Data Science — Oracle Cloud Infrastructure, exposto como endpoint REST consumido pelo Oracle APEX via ORDS.

5. Justificativa Técnica do Modelo

A escolha do modelo de séries temporais não foi arbitrária. Ela deriva diretamente das características do problema e dos dados disponíveis no projeto. A tabela abaixo apresenta a justificativa técnica comparativa:

Critério	Análise e Decisão
Natureza dos dados	Os registros de agendamentos são eventos datados com padrões temporais claros. Modelos de séries temporais capturam essas dependências de forma nativa, sem necessidade de feature engineering complexo.
Sazonalidade	A demanda hospitalar apresenta sazonalidades sobrepostas: picos nas segundas-feiras, quedas nos fins de semana, aumento em meses de inverno e variação por feriados. O Prophet foi projetado especificamente para capturar múltiplas sazonalidades simultâneas.
Por que não CNN	Redes convolucionais são ideais para dados com estrutura espacial, como imagens e sinais bidimensionais. Os dados de

	agendamento formam uma sequência unidimensional de contagens, aplicar CNN aqui não agrega valor técnico ao problema.
Por que não NLP	Processamento de linguagem natural é adequado para texto livre. O problema central desta sprint é quantitativo: queremos prever um número de atendimentos, não interpretar descrições textuais.
Interpretabilidade	O Prophet decompõe a previsão em componentes separados: tendência geral, sazonalidade semanal, sazonalidade anual e efeito de feriados. O gestor não apenas recebe o número — ele entende por que a previsão foi gerada, o que aumenta a confiança e a adoção do sistema.
Dados disponíveis	O modelo de domínio do projeto já possui a entidade Agendamento/Consulta com data, especialidade e vínculo com a unidade. Os dados necessários estão inteiramente dentro do escopo implementado nas sprints anteriores.
Ecosistema Oracle	A integração via OCI Data Science permite hospedar o modelo Python como endpoint REST gerenciado, consumível diretamente pelo Oracle APEX via ORDS, sem sair do ecossistema Oracle exigido pela disciplina.

6. Caso de Uso no Oracle APEX

O cenário abaixo descreve, passo a passo, o que acontece no sistema quando o gestor aciona a funcionalidade de Inteligência Artificial no painel Oracle APEX:

1. O gestor acessa o painel 'Previsão de Demanda' na área de Analytics do Oracle APEX.
2. Seleciona os parâmetros desejados: unidade de saúde, especialidade médica e horizonte de previsão (7, 15 ou 30 dias).
3. O APEX dispara uma requisição ao Oracle REST Data Services (ORDS), que aciona a stored procedure SP_EXPORTAR_SERIE_TEMPORAL no Oracle Database.
4. A procedure consolida o histórico de agendamentos dos últimos 90 dias, agrupa os registros por data e especialidade e serializa o resultado em formato JSON.
5. O ORDS encaminha o payload JSON ao endpoint REST do modelo de IA hospedado no OCI Data Science.
6. O modelo processa a série temporal, aplica os componentes de tendência e sazonalidade e retorna a previsão dos próximos N dias com intervalos de confiança, um limite inferior e um superior.
7. O Oracle APEX renderiza o resultado em um gráfico de linha com três séries: histórico real, previsão central (linha tracejada) e banda de confiança (área sombreada).
8. Caso a previsão central supere o limiar de capacidade configurado para aquela unidade, o sistema exibe automaticamente um alerta: 'Previsão de pico para a especialidade X no dia Y. Recomendado reforço de escala.'

7. Estratégia de Dados

7.1 Origem e Formato

Os dados que alimentam o modelo de IA são provenientes do Oracle Database, especificamente da tabela TB_AGENDAMENTO, já prevista no modelo de domínio do projeto Medix desde as primeiras sprints. A estrutura mínima necessária para o funcionamento do modelo é:

Campo	Tipo	Descrição
id	NUMBER	Identificador único do agendamento.
data_agendamento	DATE	Data em que o atendimento foi realizado ou agendado.
especialidade	VARCHAR2	Especialidade médica do atendimento (ex.: Cardiologia, Pediatria).
unidade_id	NUMBER	Chave estrangeira para TB_UNIDADE.
status	VARCHAR2	Situação do registro: Realizado, Cancelado ou Agendado.

O formato de saída da procedure, enviado ao modelo, é um array JSON com a contagem diária de atendimentos por especialidade:

```
[ { "ds": "2025-01-15", "y": 23, "especialidade": "Cardiologia",  
  "unidade_id": 1 }, ... ]
```

A resposta do modelo retorna a previsão com os limites de confiança:

```
{ "yhat": 27, "yhat_lower": 21, "yhat_upper": 33, "alerta": true }
```

7.2 Volume Mínimo e Granularidade

O modelo requer um mínimo de 90 dias de histórico por especialidade por unidade para gerar previsões confiáveis. A granularidade utilizada é diária, os registros são agrupados por data, especialidade e unidade antes de serem enviados ao modelo. Para produção, o ideal é 365 ou mais dias de histórico para capturar a sazonalidade anual completa.

7.3 Estratégia para o MVP

Para a demonstração desta sprint, será utilizada uma estratégia de dados sintéticos realistas gerados por script Python. Os dados simulam padrões típicos de uma unidade de saúde: picos nas segundas-feiras, queda nos fins de semana, sazonalidade mensal e variações por feriados. Essa abordagem valida o pipeline completo, da procedure ao gráfico no APEX, sem depender de dados reais. A substituição por dados reais ocorrerá progressivamente à medida que as unidades registram agendamentos na plataforma.

8. Diagrama de Arquitetura

O diagrama abaixo representa a comunicação entre as três camadas obrigatórias definidas no edital da disciplina. As setas indicam o fluxo bidirecional: requisição de previsão (↓) e retorno dos resultados (↑).

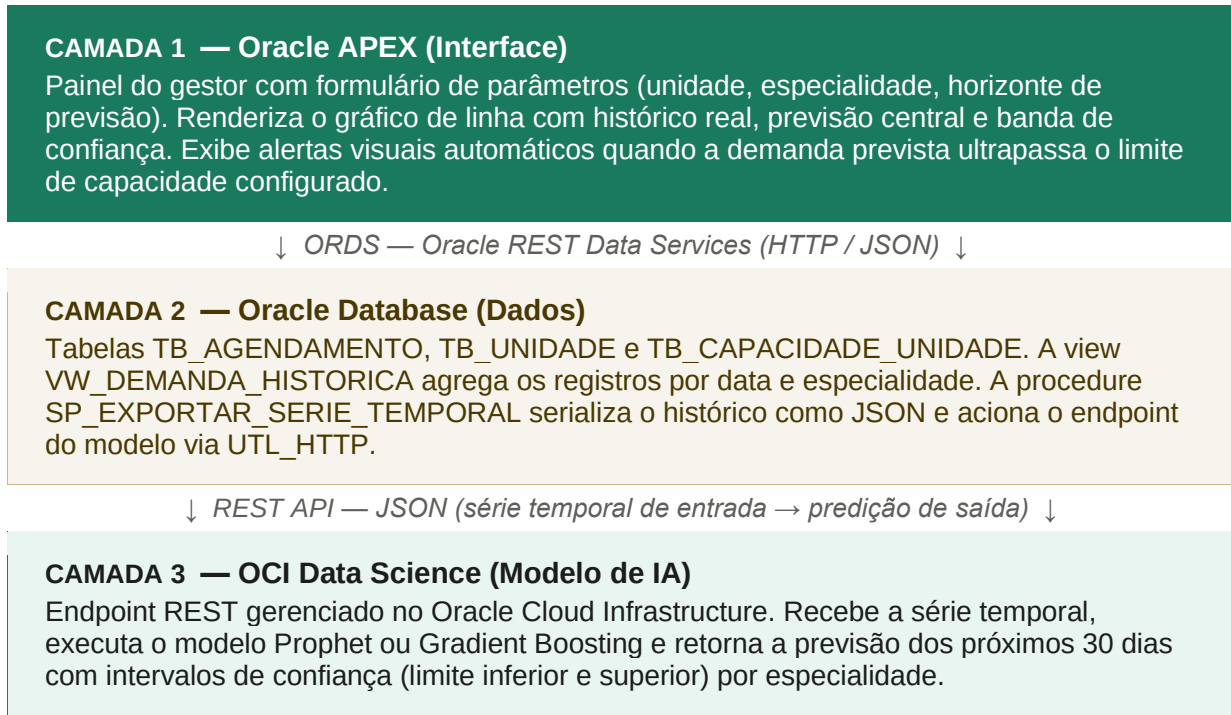


Figura 1 — Arquitetura de integração Medix AI: Oracle APEX (interface do gestor) → Oracle Database (dados e procedures) → OCI Data Science (modelo de IA).

9. Alinhamento com os Critérios de Avaliação

Critério do Edital	Como este documento atende
Aplicação Técnica — até 60 pontos	Justificativa técnica formal com comparação explícita a CNN e NLP; caso de uso detalhado em 8 etapas; estratégia de dados com origem, estrutura, formato JSON, volume mínimo e plano para MVP.
Diagrama de Arquitetura — obrigatório	Presente na Seção 8, cobrindo as três camadas exigidas: Oracle APEX, Oracle Database e Modelo de IA, com os protocolos de comunicação identificados nas setas.
Vídeo Pitch — até 20 pontos	Roteiro estruturado em 5 blocos com tempo definido, cobrindo problema, solução, justificativa técnica, arquitetura e resultados. Postado no YouTube como 'Não Listado'.
Organização e Repositório — 20 pontos	README dedicado no GitHub com instruções, tecnologias, resultados e link do vídeo. Código parcial do modelo e scripts de dados disponível no repositório.

10. Tecnologias Utilizadas

Tecnologia	Versão	Papel na Arquitetura
Oracle APEX	23.x	Interface web do gestor, formulário de parâmetros e visualização do gráfico de previsão.
Oracle Database	19c+	Armazenamento dos históricos e execução da stored procedure de exportação.
Oracle REST Data Services (ORDS)	23.x	Gateway REST que conecta o APEX ao banco e ao modelo externo.
OCI Data Science	—	Ambiente de hospedagem do modelo Python como endpoint REST gerenciado.
Prophet (Meta)	1.1.x	Modelo de séries temporais com suporte nativo a sazonalidade múltipla e feriados.
XGBoost / LightGBM	—	Alternativa de Gradient Boosting para maior precisão com volume de dados elevado.
Python	3.10+	Linguagem de treinamento e inferência do modelo (scikit-learn, pandas, numpy).
Oracle UTL_HTTP	—	Mecanismo PL/SQL para disparar chamadas HTTP ao endpoint do modelo a partir do banco.

11. Próximos Passos

- Criar o dashboard de previsão no Oracle APEX com gráfico de linha interativo e alertas automáticos de capacidade.
- Implementar o endpoint REST do modelo de IA no OCI Data Science com os scripts de treinamento e inferência.
- Desenvolver o script Python de geração de dados sintéticos para validação do pipeline completo no MVP.
- Integrar os dados reais de agendamento provenientes das unidades parceiras ao pipeline de treinamento do modelo.
- Expandir as previsões para o aplicativo mobile, exibindo ao paciente a estimativa de disponibilidade por especialidade em cada unidade.
- Avaliar a adoção do modelo alternativo (Gradient Boosting) conforme o volume de dados históricos crescer nas próximas sprints.

12. Repositórios GitHub

Todo o código desenvolvido até o momento está disponível nos seguintes repositórios:

Arquitetura de IA — Sprint 3 (esse repositório):

- <https://github.com/challengeoracle/>

Painel Administrativo — .NET:

- <https://github.com/challengeoracle/sprint-1-dotnet>

Aplicativo Mobile — Paciente:

- <https://github.com/challengeoracle/sprint-1-mobile>

Backend — Java:

- <https://github.com/challengeoracle/sprint-03-java>

Vídeo de Demonstração (YouTube):

-